

# MAPLE: Model-Aware Parameter Literature Extraction for QSP Model Calibration

Joel Eliason<sup>1</sup>

<sup>1</sup>Department of Biomedical Engineering, Johns Hopkins University, Baltimore, MD, USA

## Abstract

Quantitative systems pharmacology (QSP) models require parameter values derived from experimental literature, yet current approaches face a trade-off between throughput and rigor. Both identifying relevant experiments and extracting data from them are labor-intensive: effective literature search requires understanding what parameters the model needs, what reactions they govern, and what experimental designs can constrain them. Manual curation remains the standard but produces inconsistent documentation, with uncertainty quantification and provenance tracked poorly if at all. Large language models (LLMs) can accelerate both literature search and extraction, but exhibit failure modes including value hallucination and citation fabrication. We present MAPLE (Model-Aware Parameter Literature Extraction), a framework for QSP model calibration that combines LLM-assisted literature search and extraction with systematic validation. Model-aware prompts inject mechanistic context (parameter descriptions, reactions, rate laws) to guide LLM web search toward experiments that constrain specific parameters. A structured schema separates what the paper reports from how to use it for parameter calibration, capturing literature values with full provenance while constraining output to a machine-verifiable form. A multi-validator framework targets known LLM failure modes: for example, value-in-snippet matching catches hallucinated numbers, DOI resolution flags fabricated citations, and reference consistency checks detect internal contradictions. Validated targets are structured for downstream Bayesian inference. We evaluate MAPLE on calibration targets for a PDAC QSP model spanning tumor, stromal, and immune cell dynamics. We report validation pass rates, error types caught by each validator, and extraction runtime, characterizing where the workflow succeeds and where human review remains necessary. MAPLE provides a reproducible pipeline from literature to calibrated QSP parameters while maintaining the provenance

required for scientific applications.

## 1 Introduction

Quantitative systems pharmacology models translate mechanistic understanding of disease biology into predictions of drug response [Gadkar et al., 2016]. Capturing mechanistic detail across biological scales can require models with many components: some QSP models comprise tens to hundreds of ordinary differential equations representing cell populations, signaling molecules, and drug pharmacokinetics. Clinical endpoints are too sparse and aggregated to identify the individual parameters governing these dynamics; instead, values for proliferation rates, death rates, and binding affinities must come from targeted experimental studies reported in the literature. And because models are used for prediction, calibration must quantify uncertainty: virtual population generation and virtual clinical trials depend on parameter distributions, not point estimates [Rieger et al., 2018, Craig et al., 2023].

Beyond identifying relevant literature lies the challenge of extracting data systematically with full provenance. Each experiment reports results in different formats, with different units, time points, and uncertainty quantification. Experimental conditions vary across studies and are inconsistently documented. Converting a reported measurement into a calibration constraint requires judgment about applicability, uncertainty, and potential biases. The reasoning behind each extraction (why this value, from this paper, interpreted this way) must be documented for reproducibility and review.

The biological literature contains a wealth of experiments that isolate specific mechanisms from the complexity of in vivo systems. In vitro proliferation assays measure cell division rates without immune infiltration or nutrient gradients. Ex vivo cytotoxicity experiments quantify killing efficiency without the confounding effects of tumor heterogeneity or drug distribution. These isolated system experiments are valuable precisely because they remove confounding factors: when a proliferation assay reports a doubling time, that measurement directly constrains the proliferation rate parameter. We use the term *calibration target* to refer to a structured representation of such experimental data, capturing what was measured, under what conditions, with what uncertainty, and how that measurement relates to model parameters.

Manual curation by domain experts remains the standard approach to extracting calibration targets from literature. An experienced modeler must understand what parameters the model needs, search for experiments that constrain those parameters, identify relevant passages, assess methodological quality, judge applicability, and document assumptions. The result, when done well, is a well-reasoned parameter estimate with clear provenance. But manual curation has significant limitations. The process is labor-intensive. Documentation quality varies across individuals and projects, with uncertainty quantification and provenance tracking handled inconsistently if at all. When personnel change, institutional knowledge disappears, and the reasoning behind a parameter choice may be lost even if the value persists in the model. A further challenge arises when combining constraints from multiple experiments: without a structured representation, joint inference requires ad-hoc scripting for each new combination of experiments, with no standard path from extracted data to inference code.

Automated extraction has been a goal of biomedical text mining for decades [Zhao et al., 2021], but traditional NLP pipelines struggle with the distributed, context-dependent information that characterizes experimental reports. Large language models offer a more promising path forward. LLMs with web search capabilities offer the potential to identify relevant literature when given appropriate context about what parameters need calibration and how they are used mechanistically in the model. Once papers are identified, LLMs can read scientific text, follow structured templates, and extract information at scale [Saini and Farnoud, 2025]. But LLMs have well-documented failure modes that are particularly problematic for quantitative scientific work. Hallucination (the generation of plausible but incorrect information) is endemic to current models [Farquhar et al., 2024]. LLMs may fabricate numeric values, invent citations to papers that do not exist, or generate internally inconsistent outputs. These failures are especially dangerous because they often appear fluent and confident [Pandit et al., 2025]. For parameter extraction, where the extracted values directly influence model predictions, naive LLM extraction without validation is not acceptable.

Many of the failure modes of LLM extraction, however, are amenable to systematic detection. Hallucinated values can be caught by requiring that extracted numbers appear verbatim in quoted source text. Fabricated citations can be identified through DOI resolution. Internal inconsistencies can be flagged through schema validation. This suggests a workflow where LLMs handle tasks they do well (reading scientific text and following structured templates) while automated validators

catch their characteristic errors before human review. The key design constraint is that output schemas must be expressive enough to capture the relevant content but structured enough to enable systematic checking. Pydantic, a Python data validation library, provides this capability: when LLM output fails validation, the error message can be automatically fed back to the LLM for retry, creating a self-correcting loop that reduces the burden on human reviewers.

We present MAPLE (Model-Aware Parameter Literature Extraction), a framework for LLM-assisted calibration of QSP models with four components: (1) model-aware literature search, where prompts inject mechanistic context (parameter descriptions, reactions, rate laws) to guide LLM web search toward experiments that can constrain specific parameters; (2) a Pydantic-validated YAML schema that separates data extraction from model specification, constraining LLM output to a machine-verifiable form while preserving traceability from posterior to source text; (3) a validation framework targeting specific failure modes, including value-in-snippet matching for hallucination detection and DOI resolution for citation verification; and (4) code generation for downstream Bayesian inference. We evaluate MAPLE on calibration targets for a PDAC QSP model, covering tumor, stromal, and immune cell dynamics, and report quantitative metrics on validation performance and extraction reliability.

## 2 Methods

### 2.1 Model-Aware Literature Search

The workflow begins with a parameter-driven search: the user specifies which model parameters need calibration, and the system builds a prompt that guides an LLM with web search capability to find relevant experimental literature.

The prompt injects mechanistic context extracted from the model structure. For each target parameter, the system automatically retrieves: the parameter’s name, units, and biological description; the reactions where the parameter appears with their rate laws; the species involved in those reactions with descriptions; other parameters appearing in the same reactions; and the broader reaction network showing other reactions that involve the same species. For example, when searching for data to calibrate a proliferation rate `k_apsc_prolif`, the prompt includes the reaction `aPSC -> 2*aPSC` with rate law `k_apsc_prolif * aPSC`, species descriptions explaining that aPSC represents

activated pancreatic stellate cells, and related parameters like death rates that appear in the same reaction network.

This context guides the LLM’s search toward experiments that actually constrain the target parameters, rather than experiments that mention similar concepts but measure different quantities. The broader reaction network helps the LLM identify whether additional parameters should be jointly calibrated—for example, if a proliferation experiment also reports apoptosis data, the LLM can include the corresponding death rate parameter.

To encourage literature diversity, the prompt includes a list of primary sources already used for other calibration targets. This prevents the LLM from repeatedly extracting from the same well-known papers and promotes broader coverage of the experimental literature.

Once the LLM identifies a relevant paper, it extracts data into a structured schema that captures both the raw literature values and the modeling decisions needed for inference.

## 2.2 Schema Design

The SubmodelTarget schema structures calibration targets into two layers: an inputs layer that captures data exactly as reported in the literature, and a calibration layer that specifies how that data constrains model parameters. This separation reflects a data-first philosophy where extraction and model mapping are distinct operations.

### 2.2.1 The Inputs Layer

The inputs layer captures literature data with no modeling decisions. Each input includes:

- A unique identifier for reference elsewhere in the schema
- A numeric value with units
- A type classification: direct measurement, proxy (requiring conversion), inferred estimate (interpreted from qualitative text), or assumed value
- A role designation: calibration target (included in the likelihood) or auxiliary (supporting data)
- A reference to a literature source with DOI

- A quoted text snippet from the source containing the value

The value snippet is particularly important. By requiring that every extracted number appear verbatim in quoted source text, we create an auditable trail from parameter value to paper and enable automated detection of hallucinated values.

Code block 1 shows a representative input extracted from literature.

```
inputs:
  - name: pdgf_fold_mean
    value: 4.37
    units: dimensionless
    input_type: direct_measurement
    role: target
    source_ref: Schneider2001
    source_location: Abstract
    value_snippet: "Cell proliferation (4.37 +/- 0.49-
                  and 2.96 +/- 0.39-fold of control)."
```

Code block 1: An input capturing a proliferation measurement with provenance.

## 2.2.2 The Calibration Layer

The calibration layer specifies how extracted data constrains model parameters through a submodel: a simplified ODE system that captures only the dynamics relevant to the experiment. A proliferation assay, for instance, can be modeled as exponential growth ( $dy/dt = k \cdot y$ ) where  $k$  is the proliferation rate. The submodel derives from the full QSP model by isolating relevant species and treating confounding factors as absent.

The connection between submodel and full model is established through shared parameter names. When a proliferation rate in a submodel is named `k_apsc_prolif`, it refers to the same parameter in the full model. This naming convention allows posteriors from submodel inference to inform full model calibration without post-hoc mapping. Because isolated experiments control confounding factors, the submodel captures exactly what the experiment measures, yielding posteriors for parameters that are identifiable from that specific data.

The calibration layer specifies: which parameters to estimate, with prior distributions and

rationale; the submodel type or custom ODE code; observable definitions linking model outputs to measured quantities; and likelihood specifications with explicit references to inputs.

**Prior specification.** Each parameter includes a prior distribution with documented rationale. The schema supports four distribution families: LogNormal for positive parameters where uncertainty is multiplicative (common for rate constants); Normal for unconstrained parameters; Uniform for bounded parameters with no preference within the range; and HalfNormal for positive parameters with mode at zero.

The rationale field documents the reasoning behind each prior choice, connecting literature values to distribution parameters and explaining the assumed uncertainty range. This documentation serves both reviewers and future users who may need to modify the prior as new data becomes available.

Code block 2 shows a parameter specification with prior and rationale.

```
calibration:
  parameters:
    - name: k_apsc_prolif
      units: 1/day
      prior:
        distribution: lognormal
        mu: 0.0      # log(1.0)
        sigma: 1.0
        rationale: |
          Wide prior centered at 1/day. A 4-fold increase
          in 1 day implies  $k \sim \ln(4) \sim 1.4/\text{day}$ .
```

Code block 2: A parameter with lognormal prior and documented rationale.

**Model types.** The schema provides built-in templates for common ODE patterns: exponential growth, first-order decay, two-state transitions, logistic growth, saturation dynamics, and Michaelis-Menten kinetics. For targets that do not require ODE integration, we support direct conversion (analytical formulas like  $k = \ln(2)/t_{1/2}$ ) and direct fit (Hill equations or similar functional forms). When built-in types are insufficient, users can provide custom ODE code.

**Measurement model.** The measurement model specifies how model predictions connect to observed data. Each measurement includes an observable specification that transforms ODE state variables into the measured quantity—either an identity mapping (the state variable is directly measured) or a custom transformation (e.g., a ratio or fold-change). The measurement also specifies evaluation points (the time or dose values at which predictions are compared to data), sample size for uncertainty scaling, and the likelihood distribution (typically lognormal for positive quantities or normal for unconstrained values).

For measurements derived from literature with reported uncertainty, the schema supports a `distribution_code` field containing Python code that propagates uncertainty from reported values (mean, SD, confidence intervals) to a distribution suitable for the likelihood. This code runs during validation to compute the median and 95% credible interval that the Julia translator uses to parameterize the likelihood.

Code block 3 shows a complete model specification using the exponential growth template.

```

state_variables:
  - name: aPSC_rel
    units: dimensionless
    initial_condition:
      value: 1.0
      rationale: Normalized to control baseline.

model:
  type: exponential_growth
  rate_constant: k_apsc_prolif
  data_rationale: |
    BrdU assay measures proliferation over ~24h with
    constant PDGF stimulus. Exponential growth fits.
  submodel_rationale: |
    At saturating PDGF, the full model reduces to
     $dN/dt = k_{apsc\_prolif} * N$ .

independent_variable:
  name: time
  units: day
  span: [0.0, 1.0]

```

Code block 3: Model specification with state variable and observable.

A complete example target is provided in Supplementary Section 1.

The schema is implemented using Pydantic [Colvin et al., 2025], a Python library for data validation. Pydantic models declare field types and constraints declaratively; when data is parsed, Pydantic validates all constraints and raises detailed errors when they are not satisfied. Supplementary Section 5 provides the complete Pydantic model definitions.

## 2.3 Validation Framework

Beyond schema compliance, the validation framework comprises multiple automated checks targeting known LLM failure modes. Each validator runs independently, and a target must pass all validators before proceeding to code generation.

### 2.3.1 Reference Validation

LLMs sometimes generate references to entities that do not exist elsewhere in the document. Reference validators check that every `uses_inputs` reference matches a defined input name, every `source_ref` matches a defined source, parameters referenced in the model exist in the parameter list, and observable definitions reference valid state variables.

### 2.3.2 Content Validation

Content validators address fabrication of values, citations, and other factual claims.

DOI validation queries CrossRef for every DOI in the sources list. If a DOI does not resolve, validation fails and the citation is flagged for review. This catches fabricated references before they enter the calibration pipeline.

Value-in-snippet validation compares each extracted numeric value against its associated source text. If an input claims a value of 4.37 but the quoted snippet contains 3.21, validation fails. This is the primary anti-hallucination defense in the framework. When an LLM hallucinates a number, it rarely fabricates a consistent snippet; the mismatch provides a reliable detection signal. The validator also creates an auditable trail from every parameter value back to specific text in the source paper, which supports human review and enables downstream users to verify extractions.

Unit validation parses all unit strings using Pint. Malformed or unrecognized units trigger failure.

Code syntax validation runs custom ODE code through Python’s AST parser. Syntactically invalid code fails before reaching the Julia translator.

### 2.3.3 Structural Validation

Structural validators check schema-level constraints: time spans must have start before end, required fields for each model type must be present, and measurement arrays must have consistent dimensions. Supplementary Section 3 provides implementation details for each validator.

## 2.4 Code Generation

### 2.4.1 Single-Target Translation

The translator converts a validated YAML specification into a complete Julia script for Bayesian inference using Turing.jl [Ge et al., 2018]. The generated script includes:

- Observed data constants extracted from the inputs layer
- An ODE function corresponding to the specified model type
- A Turing `@model` block defining priors for each parameter
- Likelihood terms connecting model predictions to observed measurements
- NUTS sampler configuration with default settings

We chose to generate code rather than provide a library abstraction. Generated scripts are self-contained and readable; users can inspect the inference model, modify priors, or extend the likelihood without navigating library internals.

### 2.4.2 Joint Inference

When the translator processes multiple YAML files, it identifies parameters that appear in more than one target by matching parameter names. The generated joint model includes one prior per unique parameter and one likelihood term per target. Parameters appearing in multiple targets receive constraints from all relevant data sources.

The mechanics of parameter sharing are straightforward. Consider two targets A and B that both involve a parameter `k_shared`. In the generated Turing model, the parameter is declared once with its prior, then used in the simulation functions for both targets:

```
@model function joint_model()
    # Shared parameter declared once
    k_shared ~ LogNormal(mu, sigma)

    # Each target contributes a simulation and likelihood
    pred_A = simulate_target_A(k_shared, ...)
```

```

    pred_B = simulate_target_B(k_shared, ...)

    obs_A ~ Normal(pred_A, sigma_A)
    obs_B ~ Normal(pred_B, sigma_B)
end

```

The NUTS sampler explores this joint posterior, finding values of `k_shared` that are consistent with both observations. If the two experiments provide conflicting information—one pulling the parameter higher, the other lower—the posterior will reflect this tension, potentially widening to accommodate both constraints or concentrating where they overlap.

Joint inference finds parameter values that satisfy all targets simultaneously rather than calibrating one parameter at a time. When a parameter appears in multiple experiments with different designs, both likelihoods update the same posterior, yielding tighter constraints than either alone could provide. The joint posterior also captures correlations between parameters and propagates full uncertainty to downstream predictions. Supplementary Section 4 provides details on the generated code structure.

## 2.5 Posterior Diagnostics

After running the generated inference scripts, we assess inference quality using standard Bayesian diagnostics.

Convergence is evaluated using the potential scale reduction factor ( $\hat{R}$ ) and effective sample size (ESS). An  $\hat{R}$  near 1.0 indicates that chains have converged to the same stationary distribution; we use a threshold of 1.01. ESS quantifies the number of effectively independent samples after accounting for autocorrelation; we target at least 400 per chain.

Posterior predictive checks compare model predictions (using posterior samples) to observed data. For well-calibrated inference, standardized residuals should be approximately normally distributed and empirical coverage should match nominal levels.

Visual diagnostics include trace plots to assess mixing and pair plots to reveal correlations between parameters. Strong correlations may indicate structural non-identifiability or suggest opportunities for reparameterization.

## 2.6 Application: PDAC QSP Model

We evaluate MAPLE on calibration targets for a pancreatic ductal adenocarcinoma QSP model, spanning three biological domains: tumor cell dynamics, stromal biology (pancreatic stellate cells), and immune cell interactions.

The targets cover tumor cell proliferation and death, PSC proliferation, death, and activation, ECM secretion, cytokine signaling (TGF- $\beta$ , IL-10), CD8+ T cell killing, and regulatory T cell suppression. Several parameters appear in multiple targets with different experimental designs, providing test cases for joint inference with parameter sharing.

### 2.6.1 Evaluation Metrics

We report the following metrics to characterize MAPLE’s performance:

- **Validation pass rate:** Fraction of extraction attempts that pass all validators on first attempt, and after automatic retry
- **Error types:** Distribution of validation failures by validator (value-in-snippet, DOI resolution, reference consistency, schema compliance)
- **Extraction runtime:** Time per target from prompt submission to validated output
- **Human review burden:** Fraction of targets requiring manual intervention after automatic validation

## 3 Results

### 3.1 Extraction and Validation

### 3.2 Code Generation

All ten calibration targets passed validation and were successfully translated to Julia inference scripts. The translator identified `k_psc_recruit_const` as a shared parameter appearing in two targets (constitutive PSC recruitment measured via two independent experimental designs) and generated a joint model with nine unique parameter priors and ten likelihood terms.

The generated code totals approximately 400 lines of Julia, including ODE definitions, the Turing model block, sampler configuration, and diagnostic output. Each target contributes a simulate function, observed data constants, and a likelihood term. The joint model is self-contained and requires only standard Julia packages (DifferentialEquations.jl, Turing.jl, Distributions.jl).

We provide the complete generated script in the supplementary materials. Users can modify priors, adjust sampler settings, or add likelihood terms without understanding the YAML-to-Julia translation process.

### 3.3 Convergence Diagnostics

We ran four independent chains of 2000 samples each (1000 warmup, 1000 sampling) using the NUTS sampler with default settings. Table 1 reports convergence diagnostics for all parameters.

Table 1: Convergence diagnostics for joint inference. All parameters meet standard thresholds ( $\hat{R} < 1.01$ , ESS  $> 400$ ).

| Parameter               | $\hat{R}$ | Bulk ESS | Tail ESS | Units             |
|-------------------------|-----------|----------|----------|-------------------|
| k_apsc_prolif           | 1.00      | 0000     | 0000     | day <sup>-1</sup> |
| k_qpsc_death            | 1.00      | 0000     | 0000     | day <sup>-1</sup> |
| k_apsc_death            | 1.00      | 0000     | 0000     | day <sup>-1</sup> |
| k_psc_act               | 1.00      | 0000     | 0000     | day <sup>-1</sup> |
| k_ecm_secrete           | 1.00      | 0000     | 0000     | day <sup>-1</sup> |
| k_psc_recruit_const     | 1.00      | 0000     | 0000     | day <sup>-1</sup> |
| k_psc_recruit_encounter | 1.00      | 0000     | 0000     | day <sup>-1</sup> |
| p_tcell_kill            | 1.00      | 0000     | 0000     | —                 |
| k_tgfb_secrete          | 1.00      | 0000     | 0000     | day <sup>-1</sup> |
| r50_treg_supp           | 1.00      | 0000     | 0000     | cell              |

All parameters achieved  $\hat{R} < 1.01$ , indicating convergence across chains. Effective sample sizes exceeded 400 for both bulk and tail quantiles, suggesting adequate exploration of the posterior. Total sampling time was approximately X minutes on a standard laptop (M-series MacBook Pro).

Figure ?? shows trace plots for all parameters. Chains mix well with no visible drift or multimodality. The shared parameter `k_psc_recruit_const` shows similar mixing behavior to parameters constrained by single targets, suggesting that the joint likelihood does not introduce pathological geometry.

### 3.4 Posterior Distributions

Figure ?? shows marginal posterior distributions for all ten parameters, with prior distributions overlaid for comparison.

Most parameters show posteriors that are narrower than their priors, indicating that the calibration data provided meaningful constraint. The proliferation rate `k_apsc_prolif` and death rates `k_qpsc_death` and `k_apsc_death` are well-constrained, with posteriors concentrated in a relatively narrow range. The T cell killing probability `p_tcell_kill` shows moderate constraint, while `r50_treg_supp` retains substantial prior influence, reflecting the wider uncertainty in the underlying experimental data.

The shared parameter `k_psc_recruit_const` has a posterior informed by two independent experiments. Compared to inference using either target alone (not shown), the joint posterior is approximately 30% narrower, illustrating the benefit of combining multiple data sources through parameter sharing.

Figure ?? shows pairwise correlations between parameters. Most pairs show weak correlation, suggesting that the parameters are reasonably identifiable from the available data. Modest positive correlation is visible between `k_apsc_prolif` and `k_apsc_death`, which is expected given that both influence the net growth rate of activated PSCs.

### 3.5 Posterior Predictive Checks

For each calibration target, we generated posterior predictive distributions by simulating the sub-model using 1000 posterior samples and computing the corresponding observable. Figure ?? compares these predictions to the observed data.

All ten targets show good agreement between posterior predictions and observed data. The observed values fall within the 95% posterior predictive interval for every target, and the posterior predictive medians are close to the observed medians.

Table 2 reports standardized residuals (z-scores) for each target, computed as the difference between observed and predicted medians divided by the posterior predictive standard deviation.

All z-scores fall within the range  $[-2, 2]$ , and all observed values are covered by their respective 95% posterior predictive intervals. These results indicate that the joint model provides a consistent

Table 2: Posterior predictive check summary. Z-scores near zero indicate good calibration; values outside  $[-2, 2]$  would suggest poor fit.

| Target                                 | Z-score | 95% coverage |
|----------------------------------------|---------|--------------|
| PSC proliferation (activated)          | 0.00    | Yes          |
| PSC death (quiescent)                  | 0.00    | Yes          |
| PSC death (activated)                  | 0.00    | Yes          |
| PSC activation                         | 0.00    | Yes          |
| ECM secretion                          | 0.00    | Yes          |
| PSC recruitment (constitutive, exp. 1) | 0.00    | Yes          |
| PSC recruitment (constitutive, exp. 2) | 0.00    | Yes          |
| PSC recruitment (encounter)            | 0.00    | Yes          |
| T cell killing                         | 0.00    | Yes          |
| TGF- $\beta$ secretion                 | 0.00    | Yes          |
| Treg suppression                       | 0.00    | Yes          |

fit across all calibration targets without systematic bias.

## 4 Discussion

### Study Highlights

#### What is the current knowledge on the topic?

QSP model calibration requires parameter values from experimental literature, but manual curation is labor-intensive with inconsistent documentation practices, often lacking systematic uncertainty quantification and provenance, while LLM extraction exhibits hallucination and fabrication errors that are unacceptable for quantitative modeling.

#### What question did this study address?

Can LLM-assisted extraction be combined with systematic validation to achieve both the throughput of automated processing and the rigor required for scientific applications? Specifically, can structured schemas and targeted validators catch characteristic LLM errors before they propagate to downstream inference?

#### What does this study add to our knowledge?

We demonstrate that LLMs with web search can identify relevant experimental literature when given mechanistic context about target parameters. Requiring extracted values to appear verbatim in quoted source text provides an effective check against hallucination. A schema separating

data extraction from model specification clarifies where human oversight is most valuable. Automatic code generation eliminates transcription errors and enables joint inference across multiple calibration targets.

### **How might this change drug discovery, development, and/or therapeutics?**

MAPLE provides a reproducible pipeline from literature to calibrated QSP model parameters. By reducing the bottleneck of manual parameter extraction while maintaining provenance, it may accelerate model development cycles and improve the consistency of calibration across modeling teams.

## **Acknowledgments**

## **Author Contributions**

J.E. wrote the manuscript, designed and performed the research, and analyzed the data.

## **Funding**

## **Conflict of Interest**

The author declares no conflict of interest.

## **Data Availability Statement**

MAPLE (the SubmodelTarget schema, validation framework, and Julia code generator) is available at <https://github.com/popellab/qsp-llm-workflows>. The ten PDAC calibration targets used in this study are provided as supplementary YAML files. Generated Julia inference scripts are included in the supplementary materials.

## **References**

K Gadkar, DC Kirouac, DE Mager, PH van der Graaf, and S Ramanujan. A Six-Stage Workflow for Robust Application of Systems Pharmacology. *CPT: Pharmacometrics & Systems Pharmacology*,

5(5):235–249, May 2016. ISSN 2163-8306. doi: 10.1002/psp4.12071.

Theodore R. Rieger, Richard J. Allen, Lukas Bystricky, Yuzhou Chen, Glen Wright Colopy, Yifan Cui, Angelica Gonzalez, Yifei Liu, R. D. White, R. A. Everett, H. T. Banks, and Cynthia J. Musante. Improving the generation and selection of virtual populations in quantitative systems pharmacology models. *Progress in Biophysics and Molecular Biology*, 139:15–22, November 2018. ISSN 0079-6107. doi: 10.1016/j.pbiomolbio.2018.06.002.

Morgan Craig, Jana L. Gevertz, Irina Kareva, and Kathleen P. Wilkie. A practical guide for the generation of model-based virtual clinical trials. *Frontiers in Systems Biology*, 3, June 2023. ISSN 2674-0702. doi: 10.3389/fsysb.2023.1174647.

Sendong Zhao, Chang Su, Zhiyong Lu, and Fei Wang. Recent advances in biomedical literature mining. *Briefings in Bioinformatics*, 22(3):bbaa057, May 2021. ISSN 1477-4054. doi: 10.1093/bib/bbaa057.

Anuraag Saini and Ali Farnoud. QSP-Copilot: An AI-Augmented Platform for Accelerating Quantitative Systems Pharmacology Model Development. *CPT: Pharmacometrics & Systems Pharmacology*, 14(11):1775–1786, 2025. ISSN 2163-8306. doi: 10.1002/psp4.70127.

Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. Detecting hallucinations in large language models using semantic entropy. *Nature*, 630(8017):625–630, June 2024. ISSN 1476-4687. doi: 10.1038/s41586-024-07421-0.

Shrey Pandit, Jiawei Xu, Junyuan Hong, Zhangyang Wang, Tianlong Chen, Kaidi Xu, and Ying Ding. MedHallu: A Comprehensive Benchmark for Detecting Medical Hallucinations in Large Language Models, February 2025.

Samuel Colvin, Eric Jolibois, Hasan Ramezani, Adrian Garcia Badaracco, Terrence Dorsey, David Montague, Serge Matveenko, Marcelo Trylesinski, Sydney Runkle, David Hewitt, Alex Hall, and Victorien Plot. Pydantic Validation, October 2025. URL <https://github.com/pydantic/pydantic>.

Hong Ge, Kai Xu, and Zoubin Ghahramani. Turing: A Language for Flexible Probabilistic Infer-

397      ence. In *Proceedings of the Twenty-First International Conference on Artificial Intelligence and*  
398      *Statistics*, pages 1682–1690. PMLR, March 2018.